INDEX

NAME: GEEVAN.J.V STD.: SEC.: ROLL NO.: SUB.: OS Observation

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1	05/1/26	Linux Commands.	1	
2.	07/1/26	Text Editor.	5	
3	20/1/26	Shell Scripting.	9	
4.	27/1/26	AWK Scripting.	14	
5.	3/2/26	System Call	17	
6.	10/2/26	CPU Scheduling Algorithm.	21	
7.	17/2/26	Bankers Algorithm	25	
8.	24/2/26	IPC using Shared Memory.	27	
9.	10/3/26	Producer Consumer Problem.	29	
10	17/3/26	Memory management Technique	31	
11.	24/3/26	Page Replacement Algorithms.	33.	

EXP NO: 1 LINUX Commands.

DATE: 05/1/26.

General Purpose Commands:-

* date:

shows the current date with day, month, time and the year.

OUTPUT: wed Mar 25

* date +%.m (month number)

OUTPUT: 03.

* date +%.h (month name).

OUTPUT: Mar.

* date +%.y (last 2 digits of year)

OUTPUT: 26.

* date +%.H (hours in 24-hour format).

OUTPUT: 14.

* date +%.M (minutes).

OUTPUT: 32.

* date +%.S (seconds)

OUTPUT: 10

* echo "message"

OUTPUT: message.

* bc - l

* who - show list of users.

* whoami - show login user
* clear - clear terminal screen.

Directory - Commands:-

* pwd - shows current working directory

  OUTPUT: /home/teja

* mkdir [directory name].

  => mkdir testdir

* cd testdir (move inside a directory).
* cd.. (move back to Parent directory).
* ls (list all files and directories)

  OUTPUT: Documents Downloads testdir
* ls -l

  OUTPUT:-

  drwxr-xr-x 2 teja teja 4096 mar 25 14: 20 testdir

  -rw-r--r-- 1 teja 123 mar 25 14:10 file.txt
* rmdir testdir (delete directory).

File commands:-

* touch [file name] Create an empty file

  OUTPUT: touch file1.txt
* Cat file1.txt.

  OUTPUT: Hello world

  Welcome to linux.

* head file1.txt (display first 10 lines).

OUTPUT : Hello world
welcome to Linux.

* tail file1.txt (display last 10 lines)
* rm file1.txt (remove or delete file).
* mv [old file name] [new file name]

OUTPUT : mv file1.txt newfile.txt.

* mv [file name] [Path].

→ move file to a new location.

OUTPUT : mv newfile.txt /home/teja/Documents

* wc -l newfile.txt.
* wc -c newfile.txt.
* wc -w newfile.txt.

File/Directory Permissions :-

d - directory.
r - read.
w - write
x - execute.
- - no Permission

-rw-r--r-- : type User group other.

chmod Symbols :-

u - user.
g - group.
o - other
a - all.

chmod a+rwx file.txt

→ give read, write, execute Permiss

to all.

chmod u-rw file.txt

→ remove read and write Pe

from user

Octal method:-

0 - no Permission

1 - execute

2 - write

3 - write and execute

4 - read

5 - read and execute

6 - read and write

7 - read, write and execute

chmod 777 filename:-

000 - no permission for user / group a

other

777 - full Permission.

DATE: 07/1/26

VI Editor (Visual Editor):-

* vi filename - Open file in vi. editor.

OUTPUT: vi file.txt

≈

"file 1.txt".

* Esc → i (Insert mode)

OUTPUT :- -----INSERT----.

Hello world.

* Esc → :wq (save and Quit)
* Esc → :q! (Quit without saving).
* Esc → o (Insert in next line).
* Esc → shift + o.
* Esc → shift + i
* Esc → shift + a (Insert at end of line).
* x - delete a character.
* :e! - undo all changes.
* ctrl + r - redo.
* shift + v - select whole line.
* y - copy.

Search:-

* Esc :/word

→ search from top, Press n for next match

*Esc : - ? word

→ Search from bottom Press n for Previous match.

REPLACE:-

*Esc : %. s/Current/new/g

→ replace all occurrences in file.

* Esc : 1 s/ current/new/g.

→ replace all occurences in first line

* Esc : 2,4,8 / Current / new / g.

→ replace from line 2 to 4.

Line Numbers :-

*Esc: set nu - show line numbers.

* Esc : set nonu - remove line numbers

SED (Stream Editor):-

* sed - stream editor command.

* - e - multiple expression

* d - delete.

* g - global.

* P - Point

Print operations :-

* sed -n '3p' data.txt (Print 3rd line)

  OUTPUT : India.

* sed -n '$p' data.txt (Print last line).

  OUTPUT : End.

* sed -n '3, 5p' data.txt (Print 3rd to 5th line)

  OUTPUT : India
  USA
  UK.

Delete operations :-

* sed '4d' data.txt.

  → delete 4th line

* sed $d' data.txt.

  → delete last line.

* sed '2, 6d' data.txt

  → delete line 2 to 6.

NANO Editor :-

* Installation - sudo apt-get update -y & sudo apt-get install nano -y.

* nano demo (open file in nano)

  OUTPUT : nano demo.txt
  GNU nano 6.x demo.txt

* ctrl+o - save file
* ctrl +x - exit editor
* ctrl +k - cut line
* ctrl + u - Paste line
* Alt +u - undo.
* Alt +e - redo
* ctrl +w - search

EXP NO: 03 shell scripting.

DATE: 20/11/25.

Arithmetic operations:-

```
#!/bin/bash

echo "Enter two numbers"
 read a
 read b.
 c = `expr $a + $b`
 d = `expr $a - $b`
 e = `expr $a \* $b`
 f = `expr $a/$b`
 g = `expr $a %. $b`
 echo = "add $c"
 echo "sub $d"
 echo "mul $e".
 echo `div $f"
 echo "mod $g"
```

OUTPUT :- Enter two numbers.

10
5
add 15
Sub 5
mul 50
div 2
mod 0.

Basic Shell :-

* If - else :-

```
#!/bin/bash
use=10
if [use -gt 18]
then
    echo "Eligible"
else
    echo "Not eligible"
fi
```

OUTPUT :- Not eligible.

* If -elif - else :-

```
#!/bin/bash
country="Nepal"
if ["$country" = "India"]
then
    echo "you are from India"
elif ["$country" = "Nepal"]
then
    echo "country not matched"
fi
```

OUTPUT :- you are from Nepal.

* while loop :-

```
#!/bin/bash
count=1
num=5
```

```
while [ $count -le $num ].
do
    echo "count is $count"
    count = $((count + 1))
done
```

OUTPUT :-

count is 1

Count is 2

count is 3.

count is 4

count is 5

*Functions :-

```
#!/bin/bash
myfun() {
        echo "Hello from function"
}
myfun.
```

OUTPUT : Hello from function.

fibonacci - series :-

```
#!/bin/bash
 echo "Enter number"
 read n
 a=0
 b=1
 echo "Enter number"
```

```
echo " Fibonacci series"
for ((i=1, i<n, i++))
do
    echo $a
    c=$([a+b])
    a=$b
    b=$c.
done.
```

OUTPUT: Enter number

5

Fib Onacci series

0
1
1
2
3.

Leap Year:-

```
#!/bin/bash
echo "Enter number"
read a
m=`expr $a %4`
if [$m -eq 0]
then
    echo "Leap year"
else
    echo "Not a Leap year"
fi
```

OUTPUT :- Enter number

2024

Leap year.

Reverse Number :-

```
#!/bin/bash.
echo "Enter number"
read n
rev = 0
while [ $n -gt 0 ].
do
  d= $ ((n %10))
  rev = $ ((rev * 10+ d))
done
echo "Reversed number is $rev"
```

OUTPUT :-

Enter number

1234

Reversed number is 4321

EXP NO: 04 AWK Scripting

DATE:- 27/1/26

Find Average salary :-

```
awk'
BEGIN {sum=0; count=0}.
{
  sum = sum + $3
  count++
}
END {
    Print "Average Salary = ", sum/count
}
'data.txt
```

OUTPUT: 1 Ram 5000 5

2 Ravi 7000 6.

3 Sita 6000 4

Average Salary = 6000

Using condition (if):-

```
awk'
{ if ($3 >6000)
        Print $2, $3.
}.
'data.txt
```

OUTPUT :- Ravi 7000.

using Logical AND:-

```
awk '
BEGIN {Sum = 0; Count = 0}.
{
  if ($3 > 6000 && $4 > 4)
  {
    Sum = Sum + $3
    Count ++
  }.
}
END {
    Print "Filtered Avg salary =" .sum / Count}
' data. txt.
```

OUTPUT:-

Filtered Avg Salary = 7000

using OR conditions-

```
awk '
{
  if ($3 >= 40 || $4 >= 40)
      Print $2 - "Pass"
  else
      Print $2, "Fail"
}
' data . txt
```

OUTPUT:-

1	Ram	35	45
2	Ravi	20	30
3	Sita	50	60

Ram Pass

Ravi Fail

Sita Pass.

Create AWK script File :-

Create File :- Vi script.awk

```
BEGIN {
        Print "Processing File"
        sum = 0 .
}
{
  sum += $3.
}
END {
     Print "Total Salary =", sum
}
```

TO RUN :-

awk -f script.awk-data.txt.

OUTPUT :-

1. Ram 5000 5

2 Ravi 7000 6

3 Sita 6000 4

Processing File...

Total Salary = 18000.

EXP NO : 05 System Call

DATE :- 3/2/26

exec :-

```
#include <stdio.h>
#include <unistd.h>
int main() {
    Printf("Before execl()\n");
    execl("/bin/ls", "ls", NULL);
    Printf("After execl()\n");
    return 0;
}
```

OUTPUT :- Before execl()
file1.txt
file2.txt
script.awk.

fork :-

```
#include<stdio.h>
#include <unistd.h>
int main() {
    if (fork() == 0) {
        Printf("This is CHILD Process\n");
    } else {
        Printf("This is PARENT Process\n");
    }
    Printf("PID:%d\n", getPid());
    return 0;
}
```

OUTPUT :-

This is child process

PID : 1235

This is PARENT Process

PID : 1234.

readdir :-

```
#include <stdio.h>
#include<dirent.h>
int main(){
    DIR *dir;
    Struct *direct* entry;
    dir = opendir("."");
    if (dir == NULL){
        Print f("cannot open directory\n");
        return 1;
    }
    Printf("Files in directory:\n");
    while((entry = readdir(dir))!=NULL){
        Print f("%s\n", entry -> d_name);
    }
    close dir (dir);
    return 0;
}
```

output :- File in directory.

:-

file 1. txt

file 2. txt

script. awk

a. out

Pid :-

```
#include <stdio.h>
#include <unistd.h>
int main() {
        Printf("PID :-/. d \n", gedPid());
        Printf(" PPID :-/-d\n ", get PPid());
        return 0;
}
```

OUTPUT :-

PID :2345

PPID : 2200.

Opendir :-

```
#include<stdio.h>.
#include<dirent.h>
int main(){
        DIR* dir;
        dir = Opendir(". ");
          Printf("Failed to open directory \n");
          return 1;
    }.
```

```
    Printf("Directory opened successfully \n");
    closedir(dir);
    return 0;
}
```

OUTPUT :-

Directory opened successfully.

EXP NO: 06 CPU scheduling Algorithms

DATE: 10/2/26.

1) FCFS:-

```
#include<stdio.h>
int main(){
    int n,i;
    Printf("Enter no of Process \n");
    scanf("%d", &n);
    for(i=0; i<n; i++){
        Printf("Enter AT and BT: \n", i+1);
        scanf("%d %d");
    }
CT[0] = AT[0] + BT[0];
for(i=1; i<n; i++);
    CT[i] = AT[i] + BT[i];
}
for(i=0; i<n; i++){
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];
}
Print f("\n Avg TAT; %.2f \n", total TAT/n);
Print f("Avg WT: %2f \n", total WT/n);
return 0;
}
```

OUTPUT:-

Enter no of Process :3

Avg TAT : 6.67

Avg WT : 3.33

2) SJF:-

```
#include <stdio.h>
int main (){
    int n, i, j time = 0, completed = 0, min,
    Printf("Enter no of Process : \n");
    while (compled < n){
        min = -1;
        for (i=0; i<n; i++){
            if (AT[i] <= time && violated[i] ==0
            {
            if (min == -1 || BT[i] < BT[min]){
                min = i;
            }
        }
    }
    float totalTAT = 0, totalWT = 0;
    totalTAT += TAT[i];
    totalWT = WT[i];
    }.
Printf("\n Avg TAT = %.2f\n" totalTAT/n);
Printf("Avg WT = %.2f\n", totalWT/n);
return 0;
}.
```

OUTPUT:-

Enter no. of Process : 4

Avg TAT = 9.25

Avg WT = 5.50

3) Round Robin:-

```
#include<stdio.h>
int main(){
    int n, tq, i, time=0, Completed=0;
    Printf("Enter Process : \n");
    scanf("%d", &n);
    Printf("Enter TQ : \n");
    Scanf("%d", &tq);
    while(Completed<n){
        int done=1;
        for(i=0; i<n; i++){
            if(RT[i]>tq){
                time += tq;
                RT[i] -= tq;
            }
        }
        if(done)
            time++;
    }
    totalTAT += TAT[i];
    totalWT += WT[i];
}
```

```
Printf("total TAT/n");
Printf ("total WT/n");
return 0;
```

3.

OUTPUT :-

```
Enter Process :3
Enter TQ : 2
6-6 7
3-0 0.
```

EXP NO: 07 Bankers Algorithm

DATE: 17/2/26

```
#include <stdio.h>
int main(){
    int P,R, i, j, k;
    Printf("Enter Process : \n");
    Scan f("%d", &P);
    Printf("Enter resource types : \n");
    scanf("%d", &R);
    int alloc[P][R], max[P][R], need[P][R];
    int finish[P], safe seq[P];
    for(j=0; j<R; j++)
        need[i][j] = max[i][j] - alloc[i][j]
    for(i=0; i<P; i++)
        finish[i] = 0;
    int count = 0;
    while(count<P){
      int found = 0;
    for(i=0; i<P; i++){
        if(finish[i]==0){
            int Possible = 1;
            for(j=0; j<R; j++){
                if(need[i][j] > work[j]{
                    Possible = 0;
                    break;
}}
```

```
        if (!found)
            break;
    }
    if (count == p) {
        Printf("system is in a safe state \n");
        Printf("safe sequence : ");
        for (i=0 ; i<p ; i++)
            Printf("%d", safe_seq[i]);
    }
    else {
        Printf("NOT in safe state \n");
    }
    return 0;
}
```

OUTPUT:-

Enter process : 5

Enter resource type : 3

system is in a safe state.

safe sequence : P_1 P_3 P_4 P_0 P_2

EXP NO: 08. IPC Using Shared memory.

DATE: 24/2/26

1) receiver.c:-

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main(){
    key_t key = 1234;
    int shmid;
    shmid = shmget(key, 1024, 0666);
    char *str = (char*) shmat(shmid, (void*)
                              0, 0);
    Printf("Data read: %s\n", str);
    shmdt(str);
    return 0;
}
```

OUTPUT:-

"Hello from shared memory
Data read: Hello from shared memory.

2) sender.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
int main(){
```

```
key = ftok("key",1234);
int shmid;
shmid = shmget(key,1024,0666|IPC_CREAT);
char *str = (char*) shmat(shmid,(void*)0,0);
Printf("Enter message");
scanf("%.d[^\n]",str);
Printf("Data written : %.s\n",str);
return 0;
}.
```

OUTPUT:-

Enter message : Hello world.

Data written : Hello world.

EXP NO: 09 Producer Consumer Problem

DATE: 10/3/26

```
#include<stdio.h>
#include<stdlib.h>
int mutex=1, full=0, empty=5, x=0;
int wait(int s)
{
  return --s;
}
int signal(int s)
{
  return ++s;
}
void Producer(){
    mutex = wait(mutex);
    empty = wait(empty);
    full = signal(full);
    x++;
    Printf("\n Producer Producer item %d", x);
}
void Consumer(){
    mutex = wait(mutex);
    full = wait(full);
    empty = signal(empty);
    Printf("\n Consumers item %d", x);
    x--;
}
int main(){
    int n;
    while(1){
```

```
printf("\n Enter choice :");
scanf("%d",&n);
switch(n){
case 1;
  if (mutex == 1 && empty != 0){
     Producer();
}
break;
case 2;
  if (mutex == 1 && full != 0){
     consumer();
  }
  default;
     printf("Invalid");
  }
}
return 0;
}.
```

OUTPUT:-

Enter your choice:1
Producer Producers item :1

EXP NO:10 Memory Management Techniques.

DATE: 17/3/26

1) First Fit :-

```
#include<stdio.h>
int main(){
    int m,n;
    Print("Enter memory blocks \n");
    scanf("%d",&m);
    int blocksize[m];
    for(int i=0; i<m; i++){
        scanf("%d", &blocksize[i]);
    }
    for(int i=0; i<n; i++){
        int allocated=0;
    for(int j=0, j<m; j++){
        int fragment = blocksize[j] - process[i];
        allocated=1;
        break;
    }
    }
    if(!allocated){
        Printf("Process %d of size %d is not
            allocated");
    }
    return 0;
}
```

OUTPUT:-

Process 1 of size 212 is allocated.

2) Best Fit ;

```
#include<stdio.h>
int main(){
    int m, n;
    printf("Enter no. of blocks : \n");
    scanf("%d",&m);
    for(int i=0; i<m; i++){
        scanf("%d", &block size[i]);
    }
    for(int i=0 ; i<n; i++){
        scanf("%d ,& Process size[i]);
    }
    if (bestIndex != -1){
        Printf("Process %d of size %d is allocate
to Block %d of size %d with fragment %d \n",
    }else{
    Printf("Process %d of size %d is not allocate
\n", i+1, Process size[i]);
    }
    }
    return 0;
}
```

OUTPUT :-

Enter no. of blocks : 5

Process 2 of size 417 is allocated to Block 2 of size 500 with fragment 83.

EXPNO: 11 Page Replacement Algorithms

DATE: 24/3/26

1) FIFO:

```
#include<stdio.h>
int main(){
    int n, frames;
    Printf("Enter Pages:\n");
    scanf("%d", &n);
    int ref[n];
    for(int i=0; i<n; i++){
        int found=0;
    for(int j=0; j< frames; j++){
        if(frame[j]==ref[i]){
        found=1;
        hits ++;
        break;
    }
    }
    if(!found){
        faults++;
    }
}
Printf("Total Page Faults::%d\n", faults);
Printf("Total Page Hits::%d\n", hits);
}
```

OUTPUT:- Enter Pages: 7

Total Page Faults: 6

Total page Hits: 1

2) LRU :-

```
#include<stdio.h>
int main(){
    int n, frames;
    Printf("Enter pages : \n");
    scanf("%d", &n);
    for(int i=0; i<n ; i++){
        int found=0;
    for(int j=0 ; j<frames ; j++){
        if(frame[j]==ref[i]){
            hits++;
            counter++;
            time[j]=counter;
            found=1;
            break;
        }
    },
    if(!found){
        counter++;
        faults++;
    }
    }.
    Printf("Total Page Faults : %d\n", faults);
    Printf("Total Page Hits : %d\n", hits);
    return 0;
}.
```

OUTPUT :-

Enter Pages :7

Total Page Faults :7

Total Page Hits :0